
Introduction to Parallel Programming

Lecture: SIMD

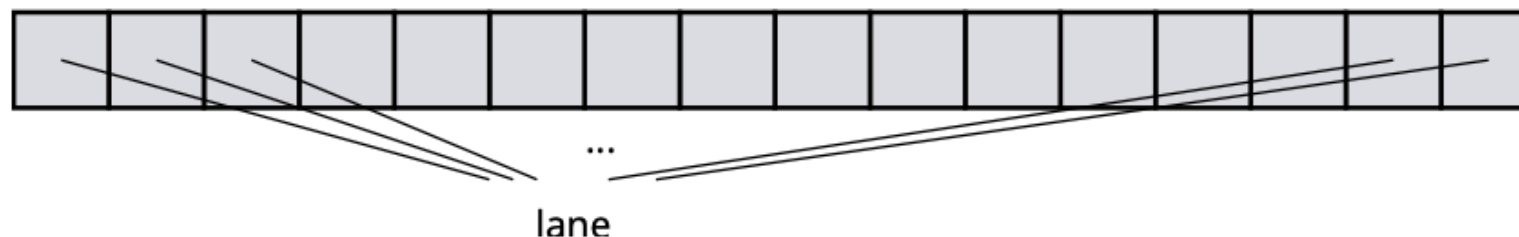
谭光明

高性能计算机研究中心

SIMD basic concepts

- **SIMD** : single instruction multiple data
- a **SIMD register** (or a **vector register**) can hold many values (2 - 16 values or more) of a single type
- a **SIMD instruction** is an instruction that can apply (typically the same) operation on all or some values on a SIMD register(s)
- each value in a SIMD register is called a SIMD lane or simply a **lane**
- they are indispensable tools for CPUs to get performance

A SIMD register



Evolving Intel instruction set

- Recent processors increasingly rely on SIMD as an energy efficient way to boost peak FLOPS

Microarchitecture	ISA	Vector width (SP)	Throughput (per clock)	max SP flops/cycle/core
Nehalem	SSE	4	1 add + 1 mul	8
Sandy Bridge	AVX	8	1 add + 1 mul	16
Haswell	AVX2	8	2 fmas	32
Ice Lake	AVX-512	16	2 fmas	64

- ISA : Instruction Set Architecture
- vector width : the number of single precision (SP) operands
- fma : fused multiply-add instruction
- e.g., Peak FLOPS of a machine having $2 \times$ Intel Xeon Gold 6130 (2.10GHz, 32 cores) = 8.6 TFLOPS
- no SIMD? $\rightarrow$ can tap at most 1/16 of SP peak performance on machines having AVX-512

Intel SIMD instructions at a glance

■ Some example AVX-512F (a subset of AVX-512) instructions

Operation	Syntax	C-like expression
multiply	<code>vmulps %zmm0,%zmm1,%zmm2</code>	<code>zmm2 = zmm1 * zmm0</code>
add	<code>vaddps %zmm0,%zmm1,%zmm2</code>	<code>zmm2 = zmm1 + zmm0</code>
fmadd	<code>vfmadd132ps %zmm0,%zmm1,%zmm2</code>	<code>zmm2 = zmm0*zmm2+zmm1</code>
load	<code>vmovups 256(%rax),%zmm0</code>	<code>zmm0 = *(rax+256)</code>
store	<code>vmovups %zmm0,256(%rax)</code>	<code>*(rax+256) = zmm0</code>

- `zmm0...zmm31` are 512 bit registers; each can hold
 - 16 single-precision (float of C; 32 bits) or
 - 8 double-precision (double of C; 64 bits)
 - floating point numbers
- `XXXps` stands for **p**acked **s**ingle precision

xmm, ymm and zmm registers

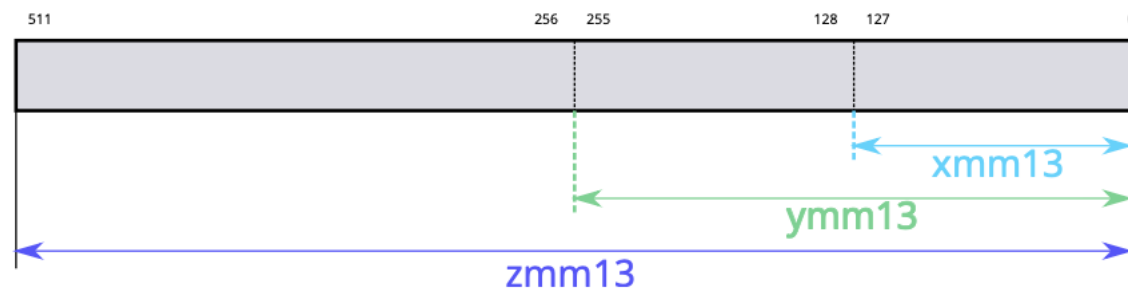
ISA and available registers

ISA	Register
SSE	xmm0, ... xmm15
AVX	{x,y}mm0, ... {x,y}mm15
AVX-512	{x,y,z}mm0, ... {x,y,z}mm31

Registers and their widths (vector widths)

Register names	Register width (bits)
xmmi	128
ymmi	256
zmmi	512

xmmi, ymmi and zmmi are aliased



Intel SIMD instructions at a glance

- look at register names (x/y/z) and the last two characters of a mnemonic (p/s and s/d) to know what an instruction operates on

Instructions	Operands	Vector/scalar?	ISA
vmulss %xmm0,%xmm1,%xmm2	1 SPs	scalar	SSE
vmulsd %xmm0,%xmm1,%xmm2	1 DPs	scalar	SSE
vmulps %xmm0,%xmm1,%xmm2	4 SPs	vector	SSE
vmulpd %xmm0,%xmm1,%xmm2	2 DPs	vector	SSE
vmulps %ymm0,%ymm1,%ymm2	8 SPs	vector	AVX
vmulpd %ymm0,%ymm1,%ymm2	4 DPs	vector	AVX
vmulps %zmm0,%zmm1,%zmm2	16 SPs	vector	AVX-512
vmulpd %zmm0,%zmm1,%zmm2	8 DPs	vector	AVX-512

- . . . **ss** : scalar single precision
- . . . **sd** : scalar double precision
- . . . **ps** : packed single precision
- . . . **pd** : packed double precision

Applications/limitations of SIMD

- SIMD is good at parallelizing computations doing almost exactly the same series of instructions on contiguous data
- $\Rightarrow$ generally, main targets are are simple loops whose index values can be easily identified

Serial Code

```
for (i = 0; i < n; i++) {  
    S(i);  
}
```

L is the SIMD width

SIMD Version

```
for (i = 0; i + L < n; i += L) {  
    S(i: i+L);  
}  
for (; i < n; i++) { /*remainder iterations*/  
    S(i);  
}
```

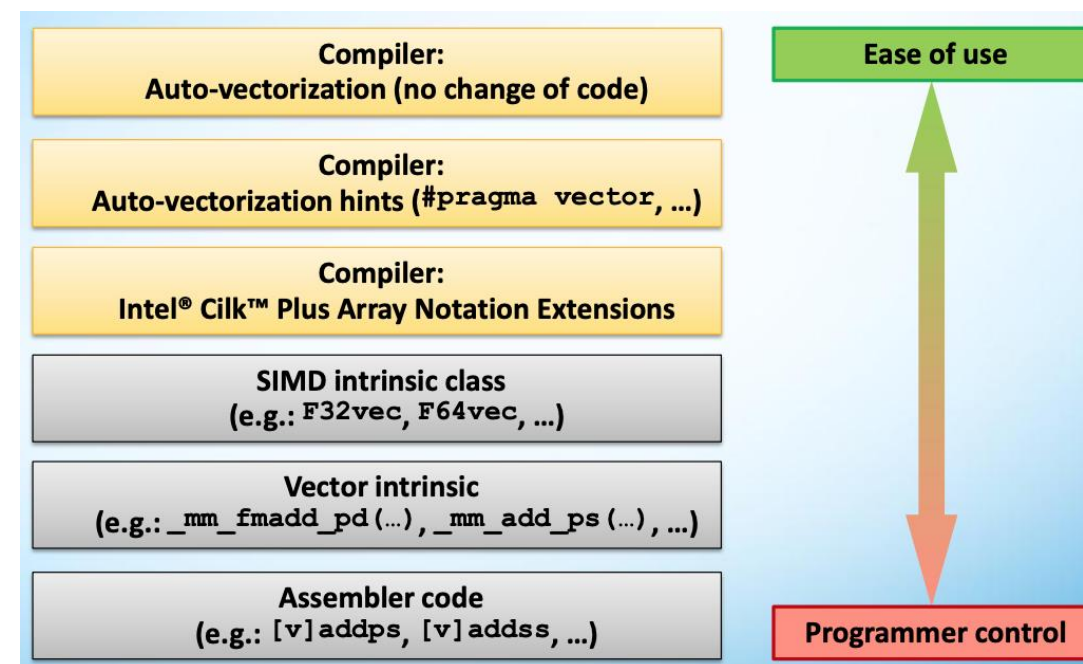
SSE vs. AVX vs. AVX-512

Instructions	SSE	AVX	AVX-512
float, double	4-way, 2-way	8-way, 4-way	16-way, 8-way
register	16x128 bits %xmm0 - %xmm15	16x256bits %ymm0 - %ymm15 The lower halves are the %xmms	32x512 bits %zmm0 - %zmm31 The lower halves are the %ymms
assembly ops	addps, mulpd, ...	vaddps, vmulpd	vaddps, vmulpd
intrinsic data type	__m128, __m128d	__m256, __m256d	__m512, __m512d
intrinsics instructions	_mm_load_ps, _mm_add_pd,	_mm256_load_ps, _mm256_add_pd	_mm512_load_ps, _mm512_add_pd

Mixing SSE and AVX may incur penalty

Several ways to use SIMD

- **auto vectorization**
 - loop vectorization
 - basic block vectorization
- **language extensions/directives for SIMD**
 - SIMD directives for loops (OpenMP 4.0/OpenACC)
 - SIMD-enabled functions (OpenMP 4.0/OpenACC)
 - array languages (Cilk Plus)
 - specially designed languages
- **vector types**
 - vector types for C/C++
 - Boost.SIMD
- **intrinsics**
- **assembly programming**



Auto loop vectorization

- write scalar loops and hope the compiler does the job

```
void axpy_auto(float a, float * x, float c, long m) {  
    for (long j = 0; j < m; j++) {  
        x[j] = a * x[j] + c;  
    }  
}
```

- Compile and run

```
linux> clang -o simd_auto -mavx512f -mfma -O3 simd_auto.c
```

- **-mavx512f -mfma** say "should use AVX-512F and FMA instructions" (better to be explicit for the time being)
- **-O3** increases the optimization level (so the compiler should work hard to vectorize it)
- read the notebook about options of other compilers (NVIDIA and GCC)

How to know if the compiler vectorized it?

- There are options useful to know whether a loop is successfully vectorized and if not, why not

compiler	Report options
clang	-R{pass,pass-missed}=loop-vectorize
NVIDIA	-M{info,neginfo}=vect
GCC	-fopt-info-vec-{optimized,missed}

- But don't hesitate to dive into assembly code
 - make -S option your friend
 - a trick: enclose loops with inline assembler comments to easily locate assembly code for the loop

```
asm volatile ("# xxxxxx loop begins");  
for (i = 0; i < n; i++) {  
    ... /* hope to be vectorized */  
}  
asm volatile ("# xxxxxx loop ends");
```

Reasons that a vectorization fails

- Potential aliasing makes auto vectorization difficult/impossible
- Complex control flows make vectorization impossible or less profitable
- Non-contiguous data accesses make vectorization impossible or less profitable
- Giving hints to the compiler sometimes (not always) addresses the problem

Aliasing and auto vectorization

- “auto” vectorizer succeeds only when the compiler can guarantee a vectorized version produces an identical result with a non-vectorized version
- Vectorization of loops operating on two or more arrays is often invalid if they point to be the same array

```
for (i = 0; i < m; i++) {  
    y[i] = a * x[i] + c;  
}
```

What if, say, `%y[i] = &x[i+1]`

- N.B., good compilers generate code that first checks `x[i:i+L]` and `y[i:i+L]` overlap
- If you know they don't overlap, you can make that explicit
- `restrict` keyword, introduced by C99, does just that

restrict keyword

- Annotate parameters of pointer type with restrict, if you know they never point to the same data

```
void axpy_auto(float a, float * restrict x, float c,
float * restrict y, long m) {
    for (long j = 0; j < m; j++) {
        y[j] = a * x[j] + c;
    }
}
```

- You need to specify -std=gnu99 (C99 standard)

```
linux> gcc -march=native -O3 -S a.c -std=gnu99 -fopt-info-vec-optimized
...
a.c:5: note: LOOP VECTORIZED.
a.c:1: note: vectorized 1 loops in function.
```

Control flows within an iteration — conditionals

- A conditional execution (e.g., if statement) within an iteration requires a statement to be executed only for a part of SIMD lanes

```
void loop_if(float a, float * restrict x, float b,  
            float * restrict y, long n) {  
    #pragma simd  
    for (long i = 0; i < n; i++) {  
        if (x[i] < 0.0) {  
            y[i] = a * x[i] + b;  
        }  
    }  
}
```

- AVX-512 supports **predicated execution (execution mask)** for that

Control flows within an iteration — nested loops

- A nested loop within an iteration causes a similar problem with conditional executions

```
void loop_loop(float a, float * restrict x, float b,  
               float * restrict y, long n) {  
    #pragma simd  
    for (long i = 0; i < n; i++) {  
        y[i] = x[i];  
        for (long j = 0; j < end; j++) {  
            y[i] = a * y[i] + b;  
        }  
    }  
}
```

- If **end** depends on **i** (SIMD lanes), it requires a predicated execution

Control flows within an iteration — function calls

- If an iteration has an unknown (not inlined) function call, almost no chance that the loop can be vectorized
 - The function body would have to be executed by scalar instructions anyways

```
void loop_fun(float a, float * restrict x, float b,  
              float * restrict y, long n) {  
    #pragma simd  
    for (long i = 0; i < n; i++) {  
        f(a, x, b, y, i);  
    }  
}
```

Non-contiguous data accesses

- Ordinary vector load/store instructions access a contiguous addresses
 - `vmovups (a), %zmm0`
 - Loads `zmm0` with the contiguous 64 bytes from address `a`
- → They can be used only when iterations next to each other access address next to each other
- That is, they cannot be used for

```
void loop_stride(float a, float * restrict x, float b, float * restrict y,
long n) {
#pragma simd
  for (long i = 0; i < n; i++) {
    y[i] = a * x[2 * i] + b; ☹️
    y[i] = a * x[i * i] + b; ☹️☹️
    y[i] = a * x[idx[i]] + b; ☹️☹️☹️
  }
}
```

AVX-512 supports `gather` instructions for such data accesses

Non-contiguous stores

■ What about store

```
void loop_fun(float a, float * restrict x, long *  
idx, float b, float * restrict y, long n) {  
    #pragma simd  
    for (long i = 0; i < n; i++) {  
        y[idx[i]] = a * x[i] + b;  
    }  
}
```

- AVX-512 supports **scatter** instructions for such data accesses
- It is your responsibility to guarantee **idx[i:i+L]** do not point to the same element

SIMD pragma

- **basic syntax (similar to omp for)**
 - c/c++: `#pragma simd [clause [, clause], ...]`

```
#pragma simd clauses
for (i = ...; i < ...; i += ...)
    Statement;
```

- **clauses**

- `vectorlength(n1 [,n2] ...)`
n1, n2, ... must be 2, 4, 8 or 16: The compiler can assume a vectorization for a vector length of n1, n2, ... to be safe Exclusive or: `vectorlengthfor(data type)`
- `private(v1, v2, ...)`
Variables private to each iteration; supersets (extensions):
 - `firstprivate(...)`: initial value is broadcast to all private instances
 - `lastprivate(...)`: last value is copied out from the last iteration instance
- `linear(v1:step1, v2:step2, ...)`
For every iteration of original scalar loop v1 is incremented by step1, ... etc. Therefore it is incremented by step1 * vector length for the vectorized loop.
- `reduction(operator:v1, v2, ...)`
Variables v1, v2, ... etc. are reduction variables for operation operator

IVDEP vs. SIMD Pragma/Directives

■ `#pragma ivdep (C/C++)`

- Ignore vector dependencies (IVDEP):
- Compiler ignores assumed but not proven dependencies for a loop

■ `#pragma simd (C/C++)`

- Aggressive version of IVDEP: Ignores all dependencies inside a loop
- It's an imperative that forces the compiler try everything to vectorize
- Efficiency heuristic is ignored
- Attention: This can break semantically correct code! However, it can vectorize code legally in some cases that wouldn't be possible otherwise!

SIMD pragma

- In general, SIMD declares “you don’t mind that the vectorized version is not the same as non-vectorized version”
- Without any clause, the directive “enforces” vectorization of the loop, ignoring all dependencies (even if they are proved!)
- Without SIMD directive, vectorization likely fails since there are too many pointer references to do a run-time check for overlapping (compiler heuristic). The compiler won’t create multiple versions here.
- Using the directive asserts the compiler that none of the pointers are overlapping.

```
void axpy_omp(float a, float * x, float c, long m) {  
    #pragma simd  
    for (long j = 0; j < m; j++) {  
        x[j] = a * x[j] + c;  
    }  
}
```

High level vectorization: summary and takeaway

- **CPUs (especially recent ones) have necessary tools**
 - arithmetic → vector arithmetic instructions
 - load → vector load and gather instructions
 - store → vector store and scatter instructions
 - if and loops → predicated executions
- **Generally, the compiler is behind CPUs; whether the compiler is able to use them is another story**
- **Become a friend of compiler reports and assembly (-S)**

Exercise

- Step-by-step Lab

Vector intrinsics

- Processor/platform-specific functions and types
- On x86 processors, put this in your code
 - `#include <x86intrin.h>`
 - and you get
 - a set of available vector types
 - a lot of functions operating on vector types
- bookmark “Intel Intrinsics Guide”
(<https://software.intel.com/sites/landingpage/IntrinsicsGuide/>)
when using intrinsics

When you want to use manual vectorization

- **Whenever your compiler fails, but in general**
 - a loop containing a branch $\Rightarrow$ predicated execution + value-blending
 - a loop accessing an array non-contiguously $\Rightarrow$ gather + scatter
 - a loop containing another loop $\Rightarrow$
 - easy if all inner loops have the same trip count
 - follow the strategy for branches (tedious)

AVX Intrinsics

- **Load and Store**
- **Constants**
- **Arithmetic**
- **Comparison**
- **Conversion**
- **Shuffles**

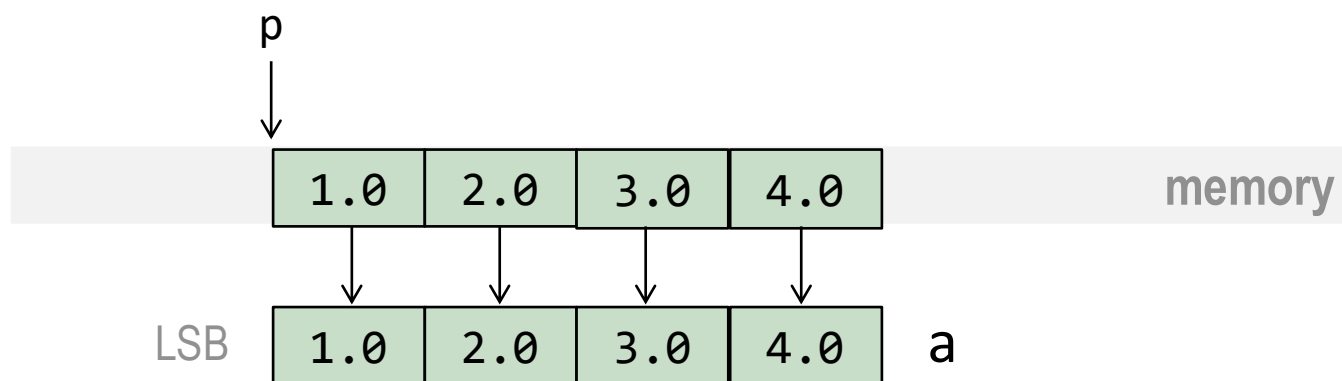
Loads and Stores

Intrinsic Name	Operation	Corresponding AVX Instructions
_mm256_load_pd	Load four double values, address aligned	VMOVAPD ymm, mem
_mm256_loadu_pd	Load four double values, address unaligned	VMOVUPD ymm, mem
_mm256_maskload_pd	Load four double values using mask	VMASKMOVPD ymm, mem
_mm256_broadcast_sd	Load one double value into all four words	VBROADCASTSD ymm,mem
_mm256_broadcast_pd	Load a pair of double values into the lower and higher part of vector.	VBROADCASTSD ymm,mem
_mm256_i64gather_pd	Load double values from memory using indices.	VGATHERPD ymm, mem, ymm

Intrinsic Name	Operation	Corresponding AVX Instructions_
_mm256_set1_pd	Set all four words with the same value	Composite
_mm256_set_pd	Set four values	Composite
_mm256_setr_pd	Set four values, in reverse order	Composite
_mm256_setzero_pd	Clear all four values	VXORPD
_mm256_set_m128d	Set lower and higher 128-bit parts	VINSERTF128

Tables show only most important instructions in category

Loads and Stores



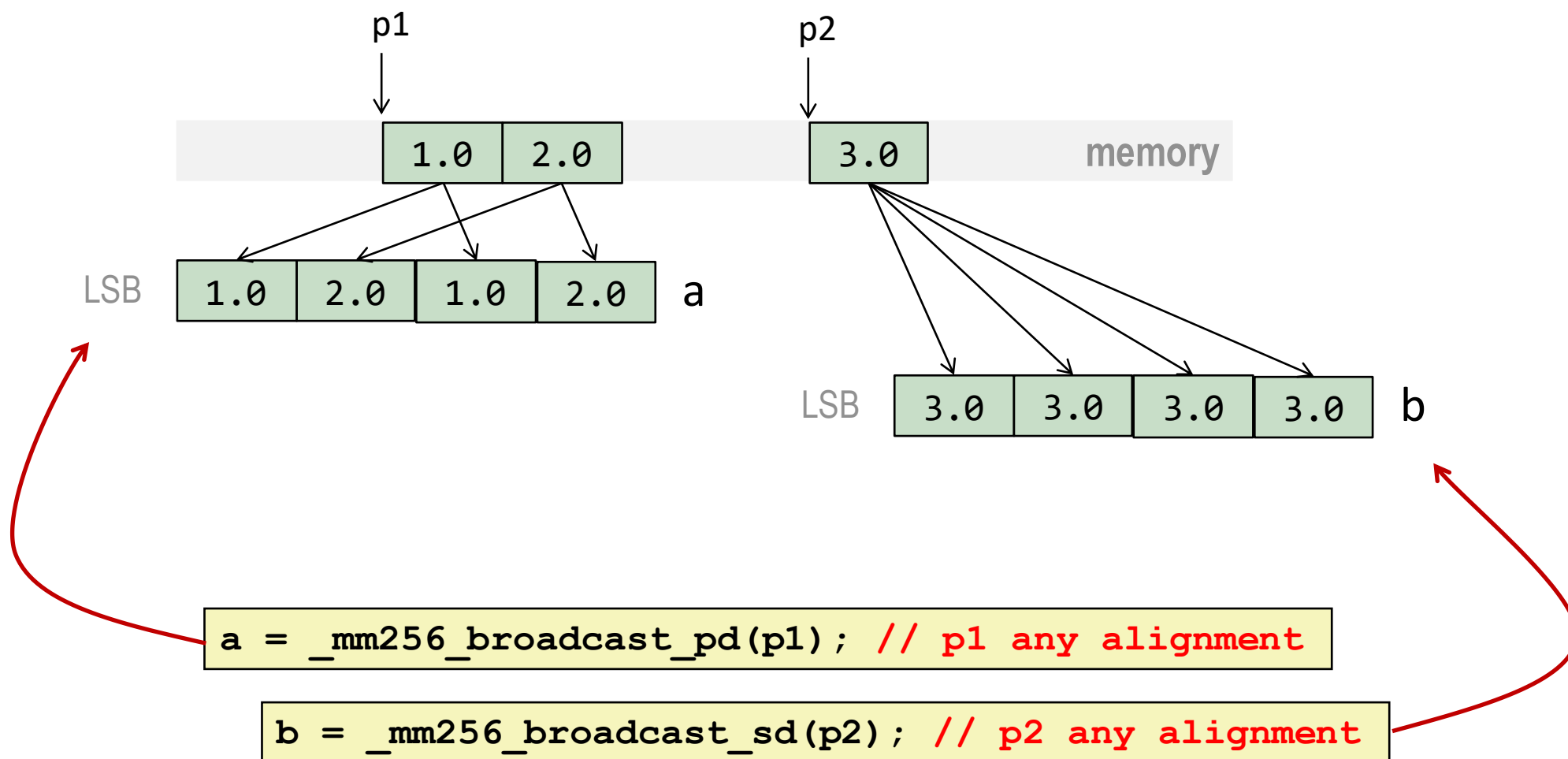
```
a = _mm256_load_pd(p); // p 32-byte aligned
```

```
a = _mm256_loadu_pd(p); // p not aligned
```

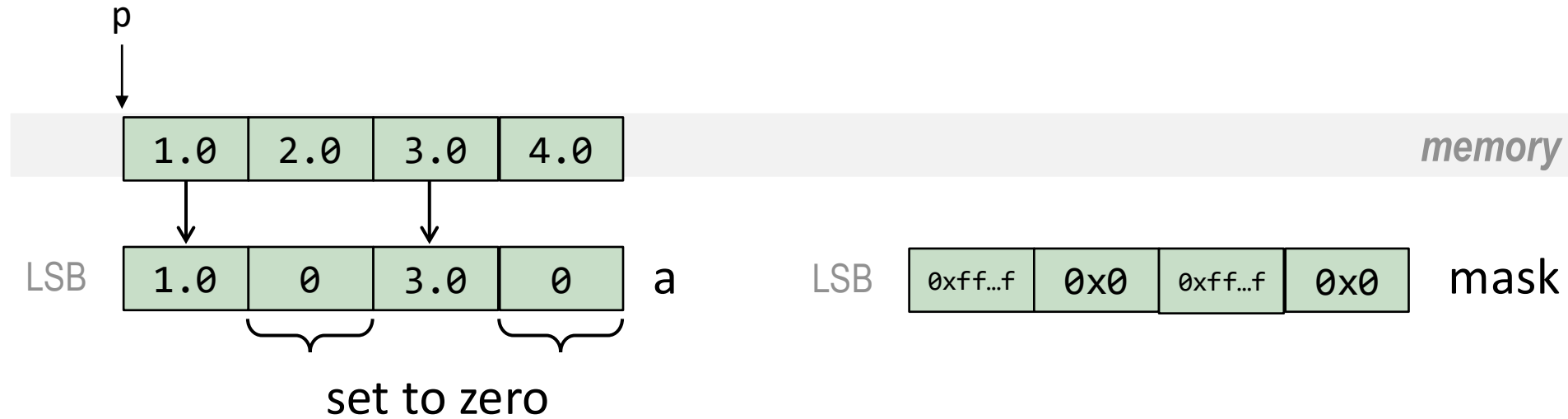
Used to be more expensive
Not anymore

`load_pd` on unaligned pointer: seg fault

Loads and Stores



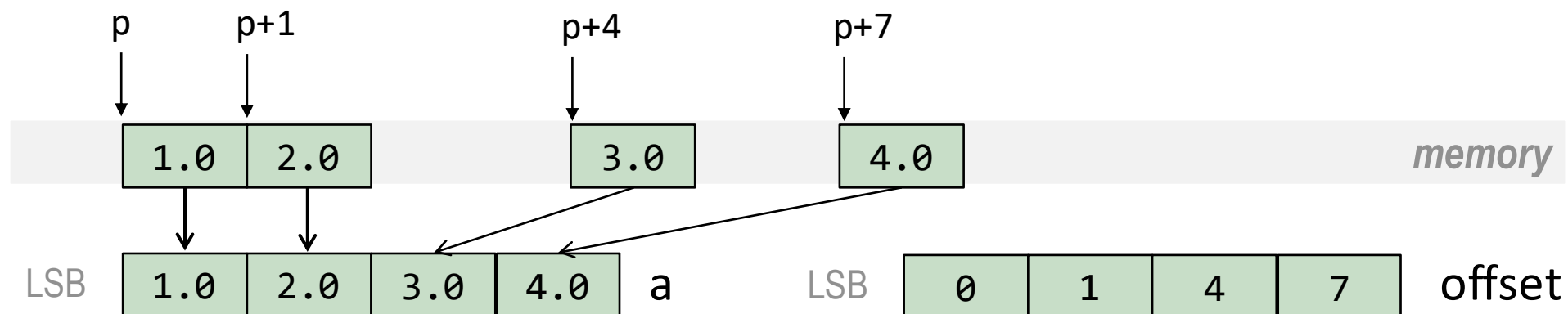
Loads and Stores



```
a = _mm256_maskload_pd(p, mask); //p any alignment
```

`__m256d`

Loads and Stores



```
a = _mm256_i64gather_pd(p, offset, 8); //p any alignment
```

Scale = {1,2,4,8}

above: scale = 8 = size of double

Stores Analogous to Loads

Intrinsic Name	Operation	Corresponding AVX Instructions
_mm256_store_pd	Store four values, address aligned	VMOVAPD
_mm256_storeu_pd	Store four values, address unaligned	VMOVUPD
_mm256_maskstore_pd	Store four values using mask	VMASKMOVDPD
_mm256_storeu2_m128d	Store lower and higher 128-bit parts into different memory locations	Composite
_mm256_stream_pd	Store values without caching, address aligned	VMOVNTPD

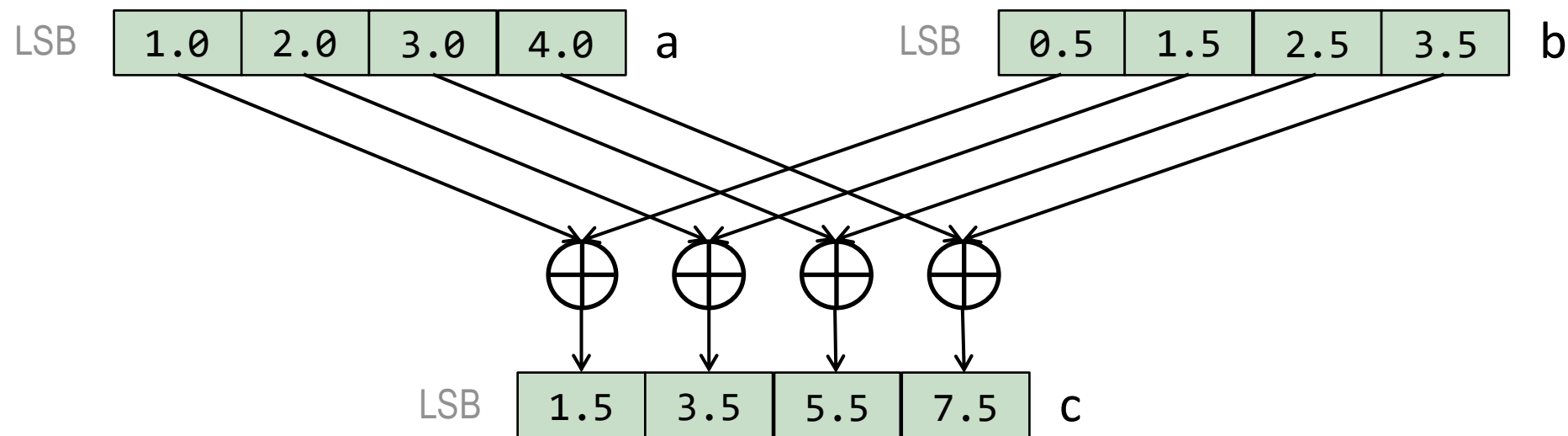
Constants

LSB	1.0	2.0	3.0	4.0	a	<code>a = _mm256_set_pd(4.0, 3.0, 2.0, 1.0);</code>
LSB	1.0	1.0	1.0	1.0	b	<code>b = _mm256_set1_pd(1.0);</code>
LSB	0	0	0	0	c	<code>c = _mm256_setzero_pd();</code>

Arithmetic

Intrinsic Name	Operation	Corresponding AVX Instructions
_mm256_add_pd	Addition	VADDPD
_mm256_sub_pd	Subtraction	VSUBPD
_mm256_addsub_pd	Alternatively add and subtract	VADDSUBPD
_mm256_hadd_pd	Half addition	VHADDPD
_mm256_hsub_pd	Half subtraction	VHSUBPD
_mm256_mul_pd	Multiplication	VMULPD
_mm256_div_pd	Division	VDIVPD
_mm256_sqrt_pd	Squared Root	VSQRTPD
_mm256_max_pd	Computes Maximum	VMAXPD
_mm256_min_pd	Computes Minimum	VMINPD
_mm256_ceil_pd	Computes Ceil	VROUNDPD
_mm256_floor_pd	Computes Floor	VROUNDPD
_mm256_round_pd	Round	VROUNDPD
_mm256_dp_ps	Single precision dot product	VDPPS
_mm256_fmadd_pd	Fused multiply-add	VFMADD132pd
_mm256_fmsub_pd	Fused multiply-subtract	VFMSUB132pd
_mm256_fmaddsub_pd	Alternatively fmadd, fmsub	VFMADDSUB132pd

Arithmetic



```
c = _mm256_add_pd(a, b);
```

analogous:

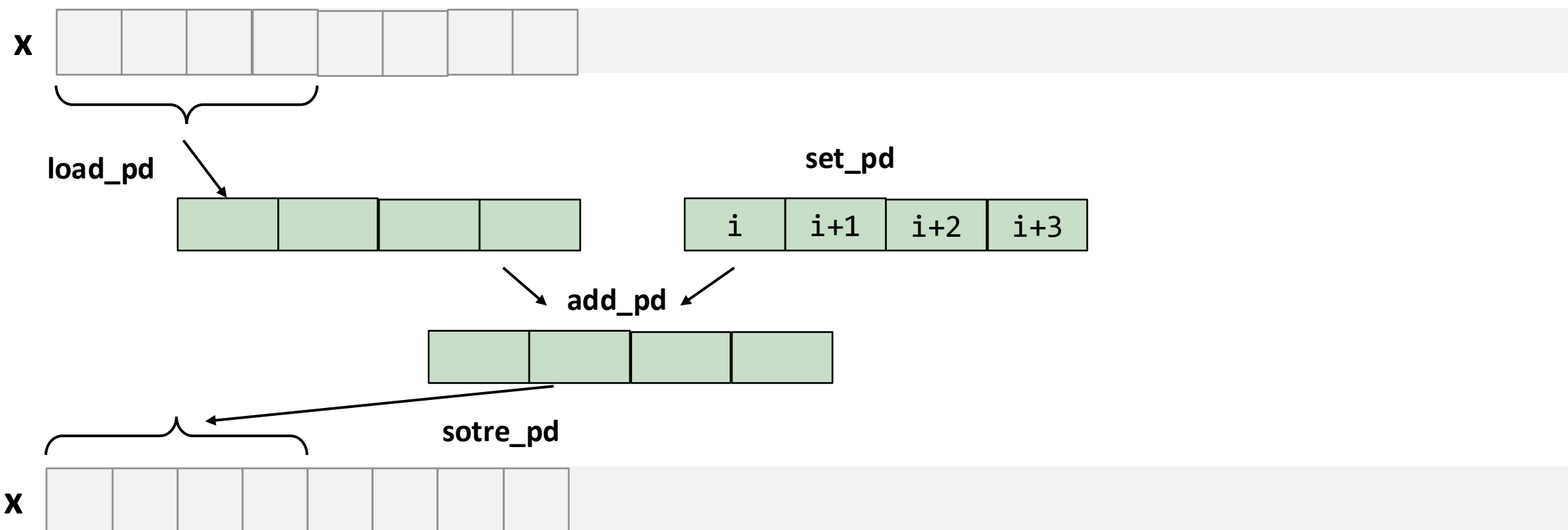
```
c = _mm256_sub_pd(a, b);
```

```
c = _mm256_mul_pd(a, b);
```

Example

```
void addindex(double *x, int n){
    for(int i = 0; i < n; i++){
        x[i] = x[i] + i;
    }
}
```

Vectorization by drawing:



Example

```
void addindex(double *x, int n){
    for(int i = 0; i < n; i++){
        x[i] = x[i] + i;
    }
}
```

```
#include <immintrin.h>

//n a multiple of 4, x is 32-byte aligned
void addindex_vec1(double *x, int n){
    __m256d index, x_vec;

    for(int i = 0; i < n; i+=4){
        x_vec = _mm256_load_pd(x+i); //load 4 doubles
        index = _mm256_set_pd(i+3, i+2, i+1, i); //create vector with indexes
        x_vec = _mm256_add_pd(x_vec, index); //add the two
        _mm256_store_pd(x+i, x_vec); //store back
    }
}
```

Is this the best solution?

No! `_mm256_set_pd` may be too expensive

Example

```
void addindex(double *x, int n){
    for(int i = 0; i < n; i++){
        x[i] = x[i] + i;
    }
}
```

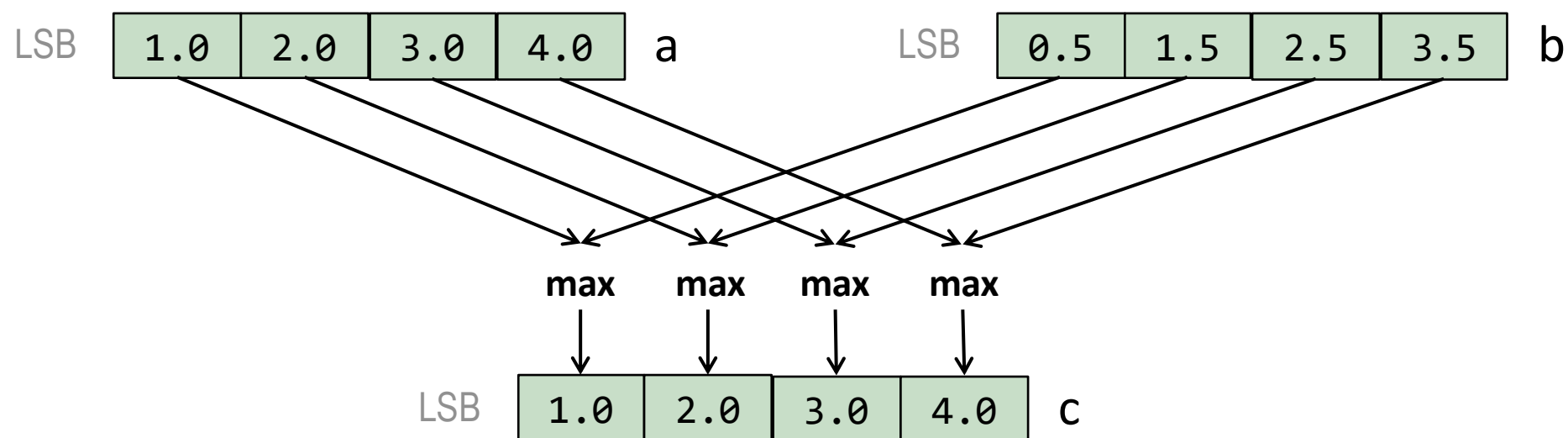
```
#include <immintrin.h>

//n a multiple of 4, x is 32-byte aligned
void addindex_vec2(double *x,int n){
    __m256d x_vec, init, incr, ind;

    ind = _mm256_set_pd(3, 2, 1, 0);
    incr = _mm256_set1_pd(4);
    for(int i=0; i < n; i+=4){
        x_vec = _mm256_load_pd(x+i); //load 4 doubles
        x_vec = _mm256_add_pd(x_vec, ind); //add the two
        ind = _mm256_add_pd(ind, incr); //update ind
        _mm256_store_pd(x+i, x_vec); //store back
    }
}
```

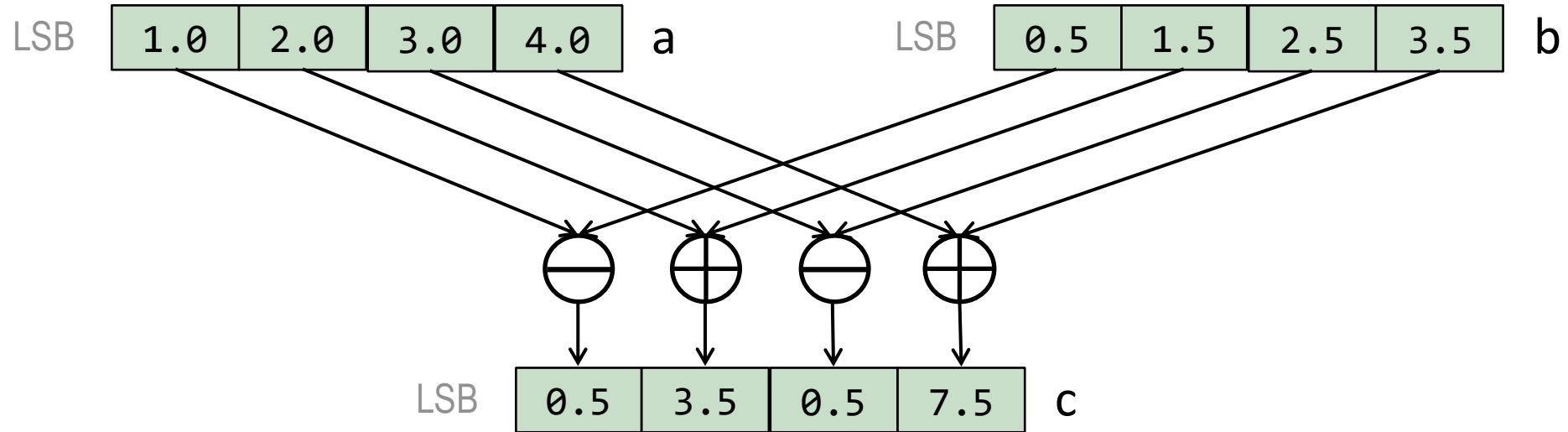
Code style helps with performance! **why?**

Arithmetic



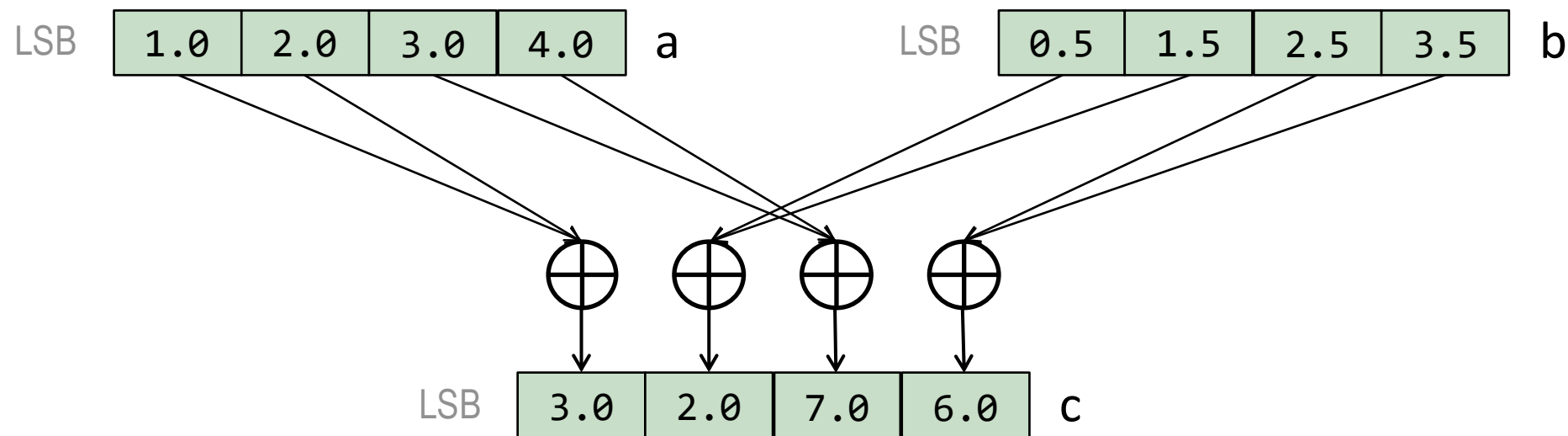
```
c = _mm256_max_pd(a, b);
```

Arithmetic



```
c = _mm256_addsub_pd(a, b);
```

Arithmetic



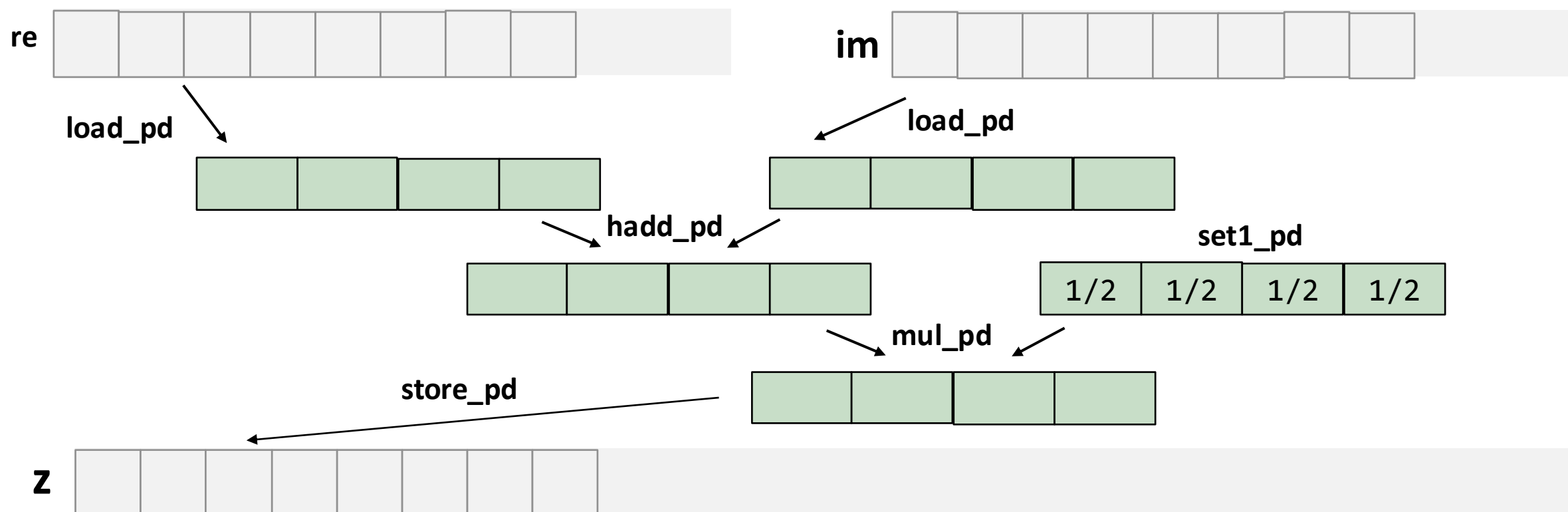
```
c = _mm256_hadd_pd(a, b);
```

analogous:

```
c = _mm256_hsub_pd(a, b);
```

Example

```
//n is even ,low pass filter on complex numbers
//output z is in interleaved format
void clp(double *re, double *im, double *z, int n){
    for(int i = 0; i < n; i+=2){
        z[i]    = (re[i] + re[i+1])/2;
        z[i+1] = (im[i] + im[i+1])/2;
    }
}
```



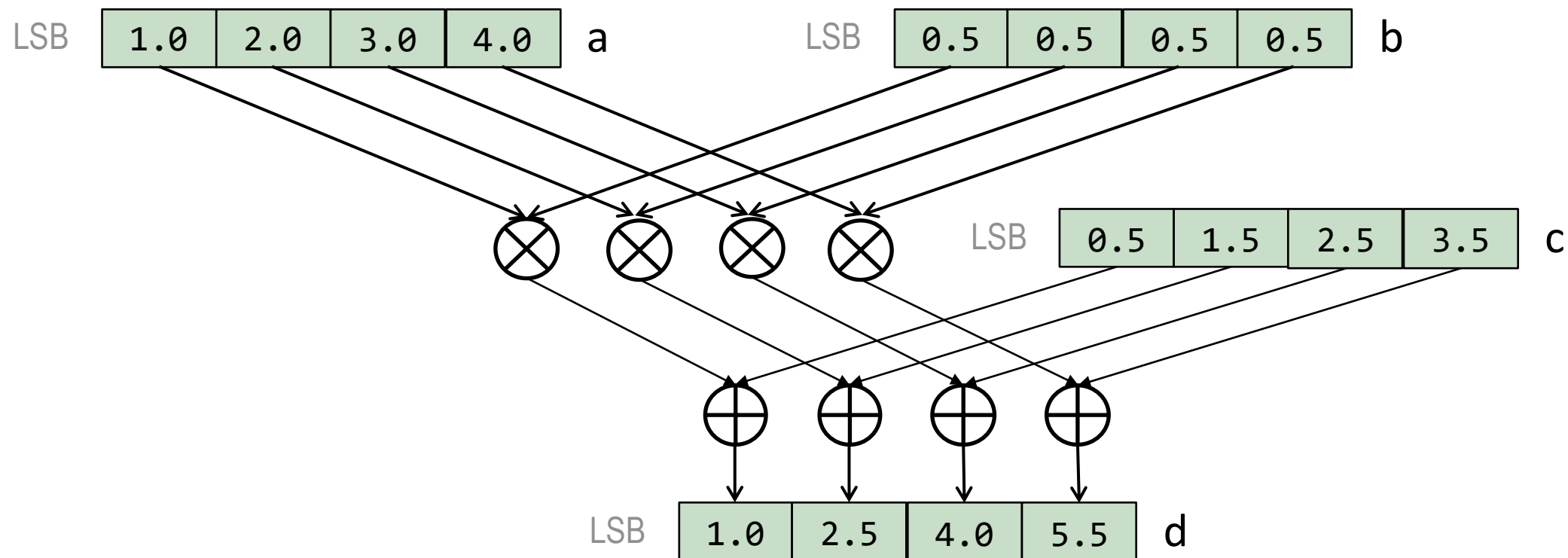
Example

```
//n is even ,low pass filter on complex numbers
//output z is in interleaved format
void clp(double *re, double *im, double *z, int n)
{
    for(int i = 0; i < n; i+=2){
        z[i]    = (re[i] + re[i+1])/2;
        z[i+1] = (im[i] + im[i+1])/2;
    }
}
```

```
#include <immintrin.h>

// n a multiple of 4, re,im,z are 32-byte aligned
void clp_vec(double *re,double *im, double *z, int n){
    __m256d half, v1, v2, avg;
    half = _mm256_set1_pd(0.5);           //set vector to all 0.5
    for(int i = 0;i < n;i+=4){
        v1  = _mm256_load_pd(re+i);       //load 4 doubles of re
        v2  = _mm256_load_pd(im+i);       //load 4 doubles of im
        avg = _mm256_hadd_pd(v1, v2);     //add pairs of doubles
        avg = _mm256_mul_pd(avg, half);   //multiply with 0.5
        _mm256_store_pd(z+i, avg);        //save result
    }
}
```

Arithmetic (FMA)



```
d = _mm256_fmadd_pd(a, b, c);
```

analogous:

```
d = _mm256_fmsub_pd(a, b, c);
```

scalar FMA:

```
d = _mm256_fmadd_sd(a, b, c);
```

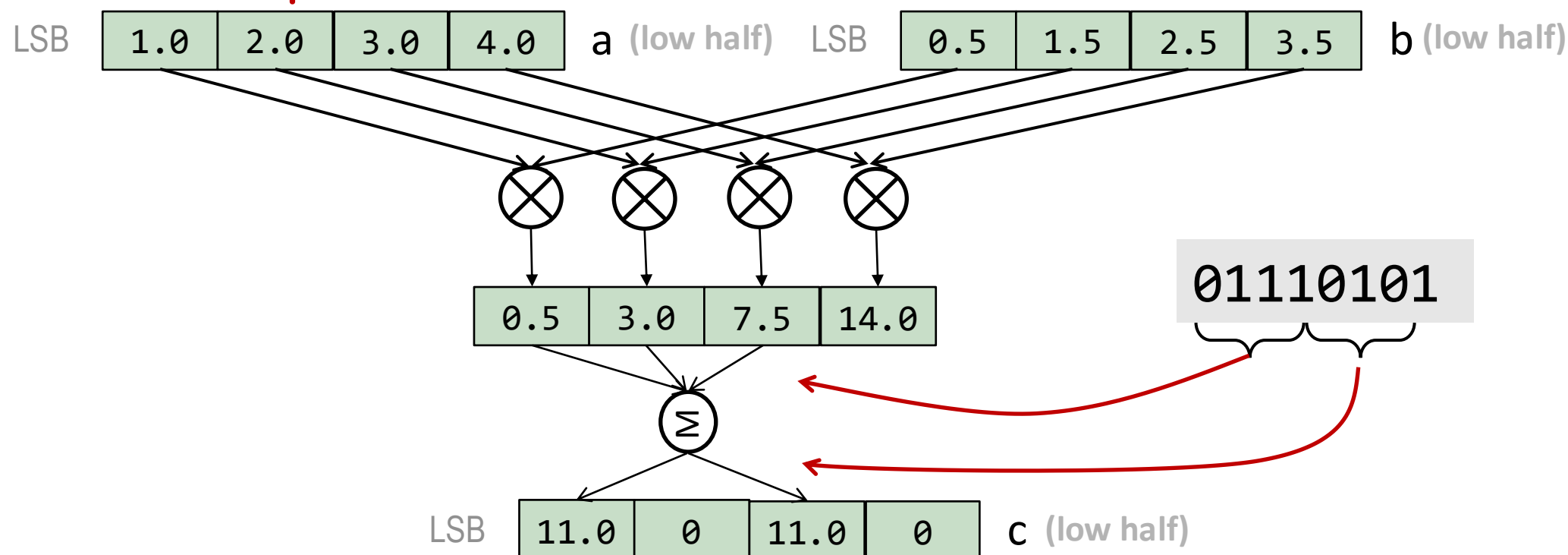
Arithmetic

```
__m256 __mm256_dp_ps(__m256 a, __m256 b, const int mask)
```

__mm256_dp_pd does not exist

Computes the pointwise product of a and b and writes a selected sum of the resulting numbers into selected elements of c; the others are set to zero. The selections are encoded in the mask. (Only for floats)

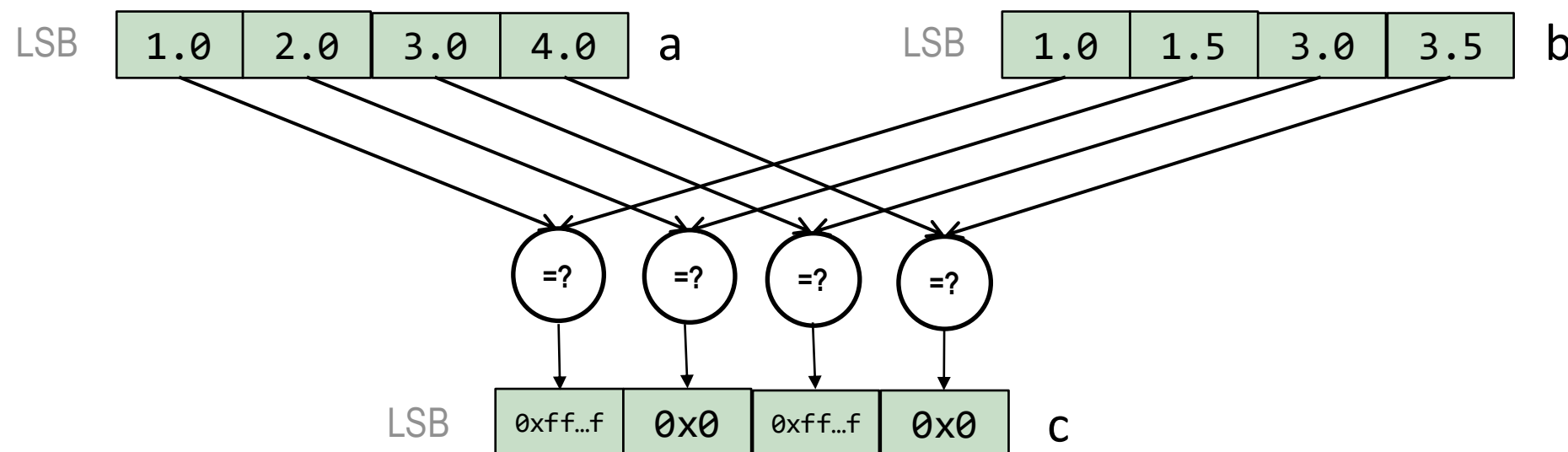
Example: mask = 117 = 01110101



Comparisons

Intrinsic Name	Macro for operation	Operation
_mm256_cmp_pd (VCMPPD)	_CMP_EQ_OQ	Equal
	_CMP_EQ_UQ	Equal(unordered)
	_CMP_GE_OQ	Greater Than or Equal
	_CMP_GT_OQ	Greater Than
	_CMP_LE_OQ	Less Than or Equal
	_CMP_LT_OQ	Less Than
	_CMP_NEQ_OQ	Not Equal
	_CMP_NEQ_UQ	Not Equal (unordered)
	_CMP_NGE_UQ	Not Greater Than or Equal (unordered)
	_CMP_NGT_UQ	Not Greater Than (unordered)
	_CMP_NLE_UQ	Not Less Than or Equal (unordered)
	_CMP_NLT_UQ	Not Less Than (unordered)
	_CMP_TRUE_UQ	True (unordered)
	_CMP_FALSE_UQ	False
	_CMP_ORD_UQ	Ordered
	_CMP_UNORD_UQ	Unordered

Comparisons



```
c = _mm256_cmp_pd(a, b, _CMP_EQ_OQ);
```

analogous:

```
c = _mm256_cmp_pd(a, b, _CMP_GE_OQ);
```

```
c = _mm256_cmp_pd(a, b, _CMP_LT_OQ);
```

etc.

Each field:
0xffff...f if true
0x0 if false

Return type: __m256d

Example

```
void fcond(double *x, size_t n){
    int i;

    for(i = 0; i < n; i++){
        if(x[i] > 0.5)
            x[i] += 1.;
        else x[i] -= 1.;
    }
}
```

```
void fcond_vec1(double *x, size_t n){
    int i;
    __m256d vt, vmask, vp, vm, vr, ones, mones, thresholds;

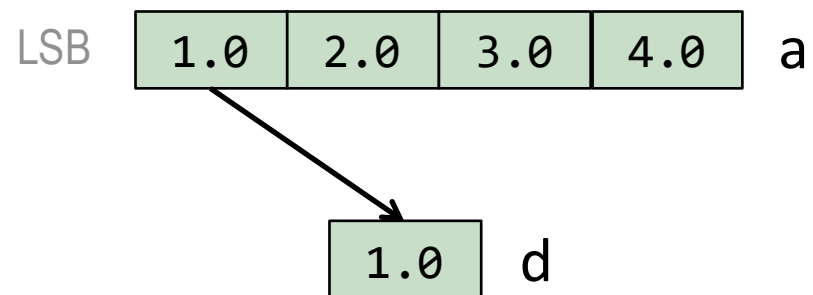
    ones      = _mm256_set1_pd(1.);
    mones     = _mm256_set1_pd(-1.);
    thresholds = _mm256_set1_pd(0.5);
    for(i = 0; i < n; i+=4){
        vt      = _mm256_load_pd(x+i);
        vmask = _mm256_cmp_pd(vt, thresholds, _CMP_GT_OQ);
        vp     = _mm256_and_pd(vmask, ones);
        vm     = _mm256_andnot_pd(vmask, mones);
        vr     = _mm256_add_pd(vt, _mm256_or_pd(vp, vm));
        _mm256_store_pd(x+i, vr);
    }
}
```

Conversion

Intrinsic Name	Operation	Corresponding AVX Instructions
_mm256_cvtepi32_pd	Convert from 32-bit integer	VCVTDQ2PD
_mm256_cvtepi32_ps	Convert from 32-bit integer	VCVTDQ2PS
_mm256_cvtpd_epi32	Convert to 32-bit integer	VCVTPD2DQ
_mm256_cvtps_epi32	Convert to 32-bit integer	VCVTPS2DQ
_mm256_cvtps_pd	Convert from floats	VCVTPS2PD
_mm256_cvtpd_ps	Convert to floats	VCVTPD2PS
_mm256_cvttpd_epi32	Convert to 32-bit integer with truncation	VCVTPD2DQ
_mm256_cvtsd_f64	Extract	MOVSD
_mm256_cvtss_f32	Extract	MOVSS

Conversion

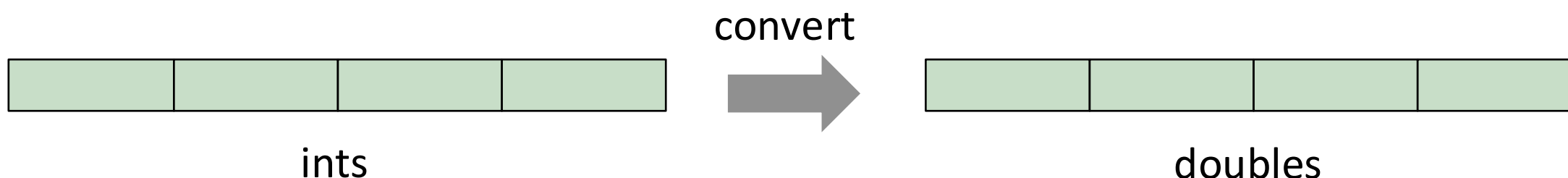
```
double _mm256_cvttsd_f64 (__m256d a)
```



```
double d;  
  
d = _mm_cvttsd_f64(a);
```

Conversion

```
__m256d _mm256_cvtepi32_pd(__m128i a)
```



See also:

```
__m256d _mm256_cvtepi64_pd(__m256i a)
```

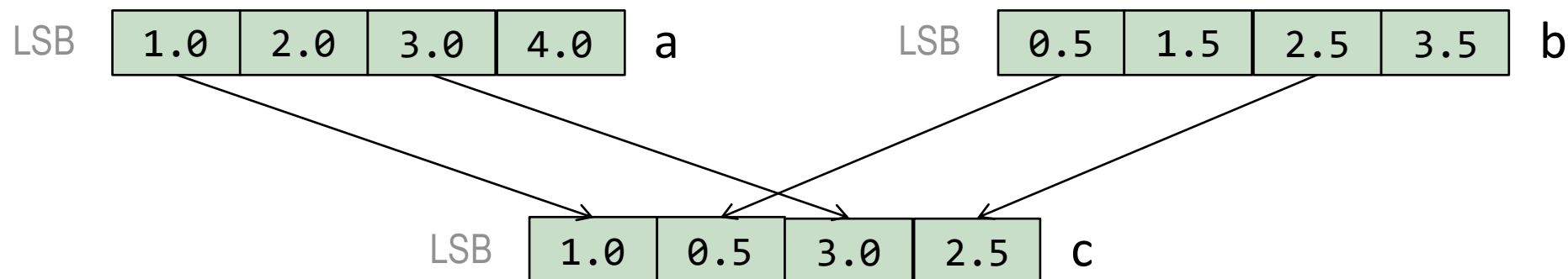
```
__m256d _mm256_cvtepu32_pd(__m256i a)
```

```
__m256d _mm256_cvtepu64_pd(__m256i a)
```

Shuffles

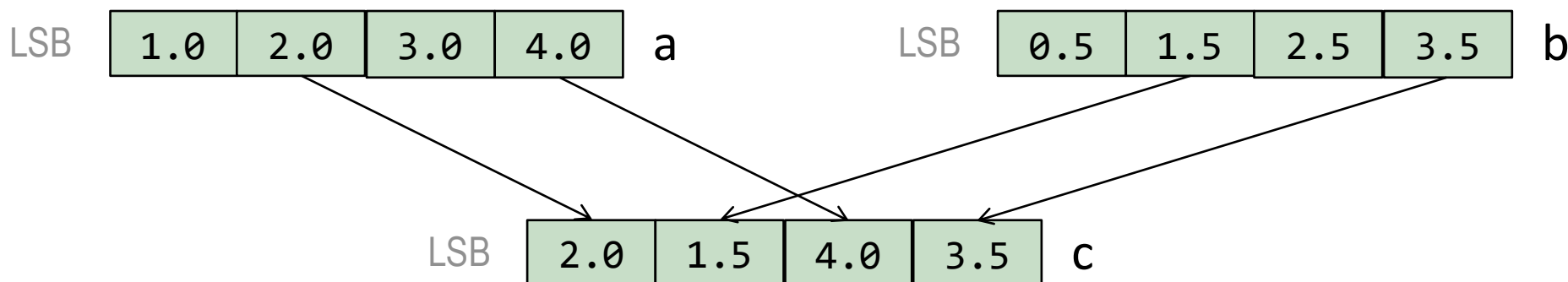
Intrinsic Name	Operation	Corresponding AVX Instructions
_mm256_unpackhi_pd	Unpack High	VUNPCKHPD
_mm256_unpacklo_pd	Unpack Low	VUNPCKLPD
_mm256_movemask_pd	Create four-bit mask	VMOVMSKPD
_mm256_movedup_sd	Duplicates	VMOVDDUP
_mm256_blend_pd	Selects data from 2 sources using constant mask	VBLENDPD
_mm256_blendv_pd	Selects data from 2 sources using variable mask	BLENDVPD
_mm256_insertf128_pd	Insert 128-bit value into packed array elements selected by index.	VINSERTF128
_mm256_extractf128_pd	Extract 128-bits selected by index.	VEXTRACTF128
_mm256_shuffle_pd	Shuffle	VSHUFPD
_mm256_permute_pd	Permute	VPERMILPD
_mm256_permute4x64_pd	Permute 64-bits elements	VPERMPD
_mm256_permute2f128_pd	Permute 128-bits elements	VPERM2F128

Shuffles



```
c = _mm256_unpacklo_pd(a, b);
```

Does not cross between
128-bit lanes



```
c = _mm256_unpackhi_pd(a, b);
```

Shuffles

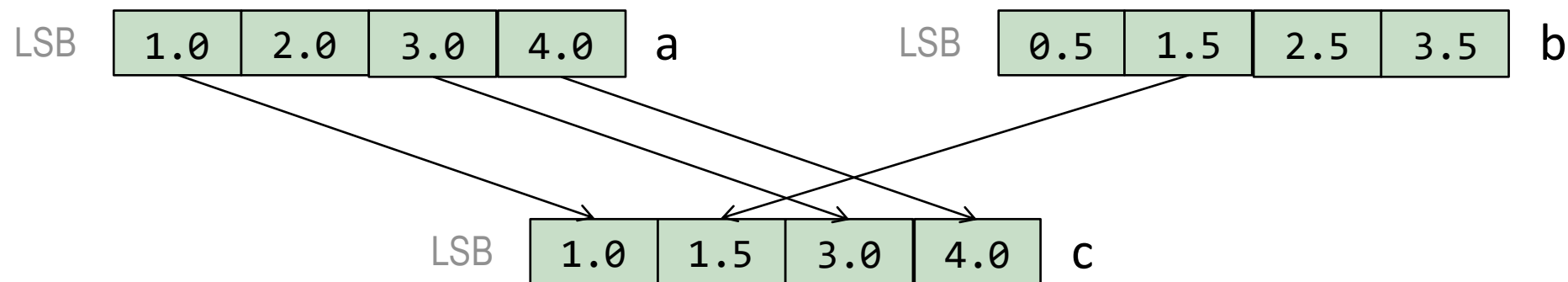
```
__m256d _mm256_blendv_pd(__m256d a, __m256d b, __m256 mask)
```

Result is filled in each position by an element of a or b in the same position as specified by mask

Example: LSB

0x0	0xff...f	0x0	0x0
-----	----------	-----	-----

 mask



Example (Continued from Before)

```
void fcond(double *x, size_t n){
    int i;

    for(int i = 0; i < n; i++){
        if(x[i] > 0.5)
            x[i] += 1.;
        else x[i] -= 1.;
    }
}
```

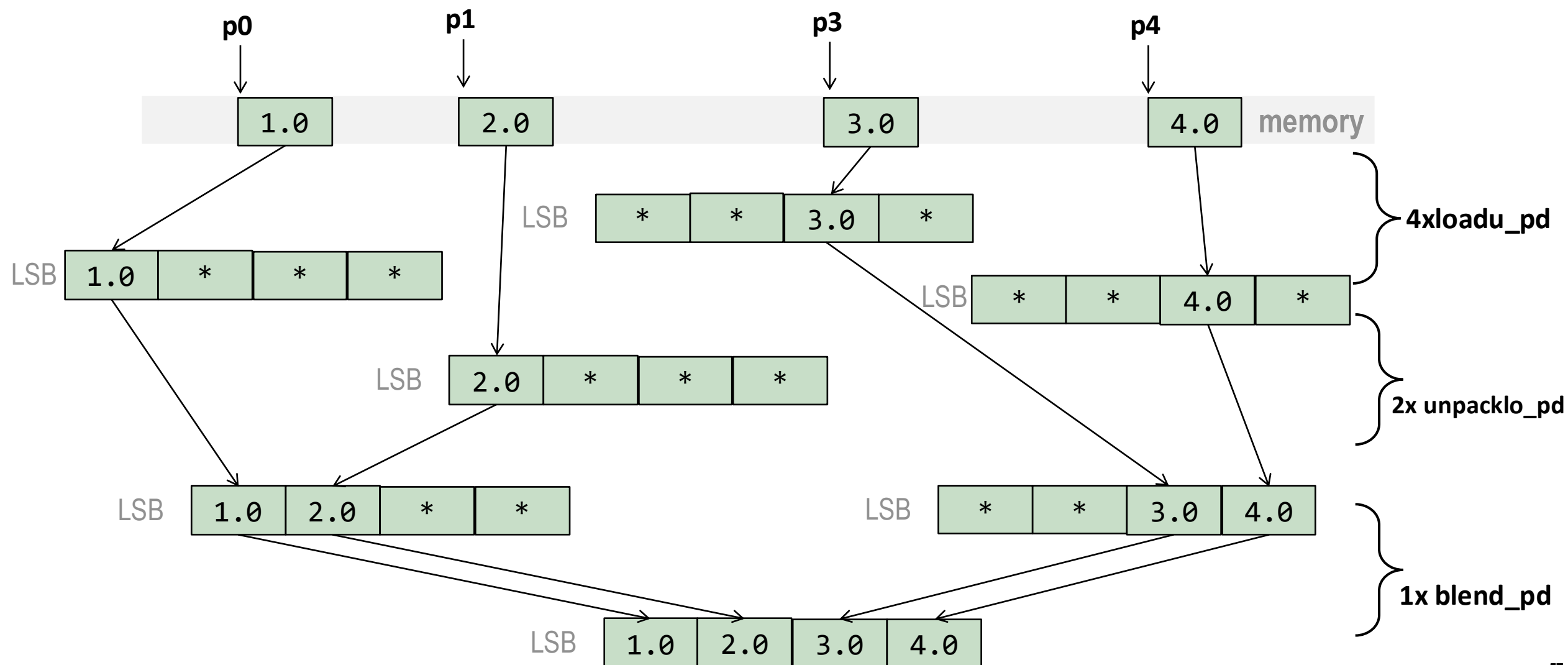
```
#include <immintrin.h>

void fcond_vec2(double *x, size_t n){
    int i;
    __m256d vt, vmask, vp, vm, vr, ones, mones, thresholds;

    ones      = _mm256_set1_pd(1.);
    mones     = _mm256_set1_pd(-1.);
    thresholds = _mm256_set1_pd(0.5);
    for(i = 0; i < n; i+=4){
        vt      = _mm256_load_pd(x+i);
        vmask   = _mm256_cmp_pd(vt, thresholds, _CMP_GT_OQ);
        vb      = _mm256_blendv_pd(mones, ones, vmask);
        vr      = _mm256_add_pd(vt, vb);
        _mm256_store_pd(x+i, vr);
    }
}
```

Example: Loading 4 Real Numbers from Arbitrary Memory Locations

Assumes all values are within one array



Code For Previous Example

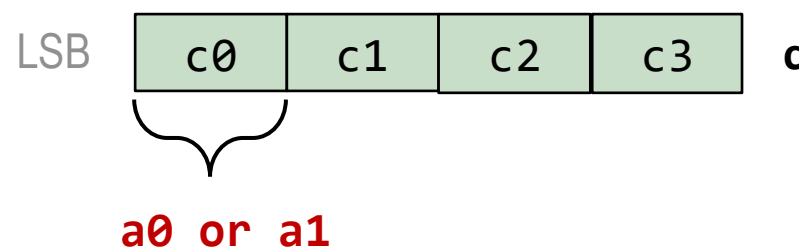
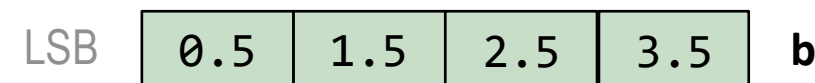
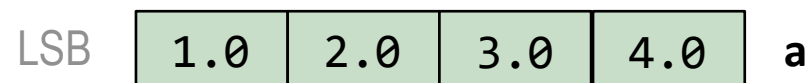
```
#include <immintrin.h>

__m256d LoadArbitrary(double *p0, double *p1, double *p2, double *p3){
    __m256d a, b, c, d, e, f;

    a = _mm256_loadu_pd(p0);
    b = _mm256_loadu_pd(p1);
    c = _mm256_loadu_pd(p2-2);
    d = _mm256_loadu_pd(p3-2);
    e = _mm256_unpacklo_pd(a, b);
    f = _mm256_unpacklo_pd(c, d);
    return _mm256_blend_pd(e, f, 0b1100);
}
```

Shuffles

```
__m256d _mm_shuffle_pd(__m256d a, __m256d b, int mask)
```



```
c0 = mask.bit0 ? a1 : a0
c1 = mask.bit1 ? b1 : b0
c2 = mask.bit2 ? a3 : a2
c3 = mask.bit3 ? b3 : b2
```

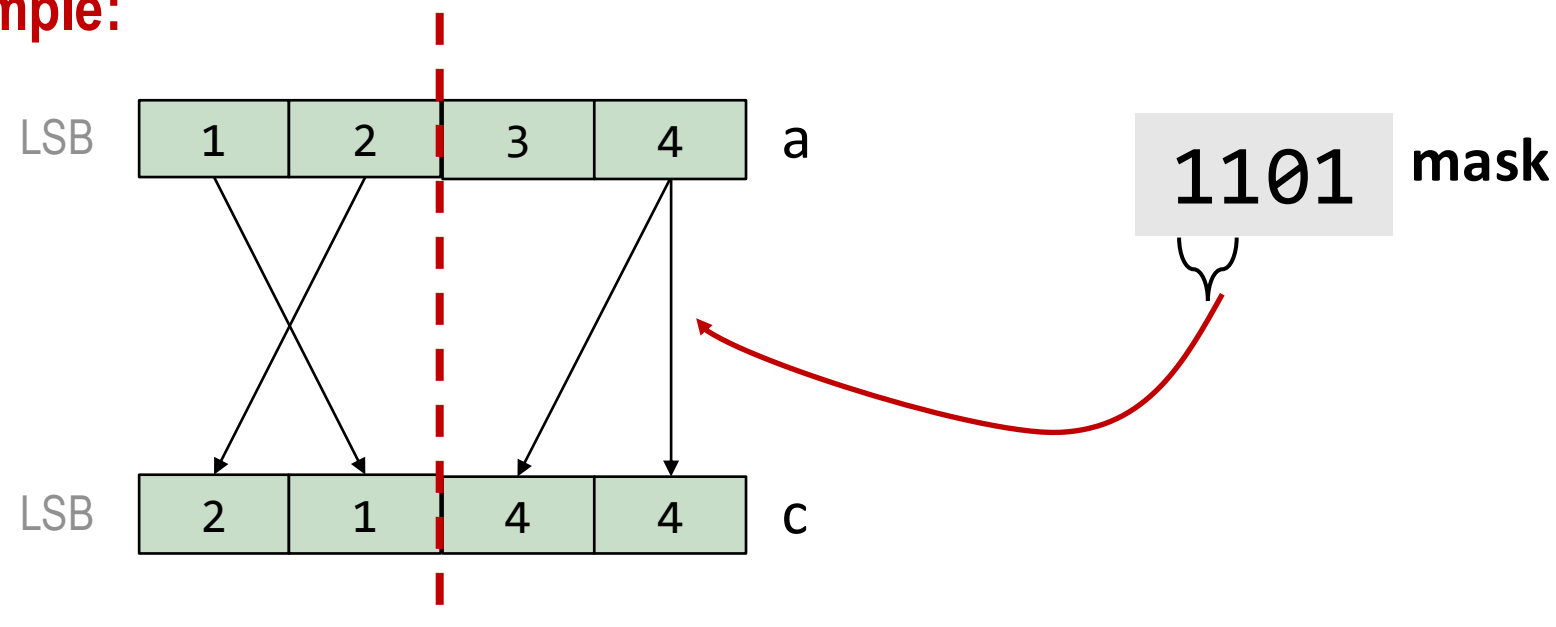
Does not cross between 128-bit lanes

Shuffles

```
__m256d _mm256_permute_pd(__m256d a, int mask)
```

Shuffle elements within 128-bits lanes

Example:



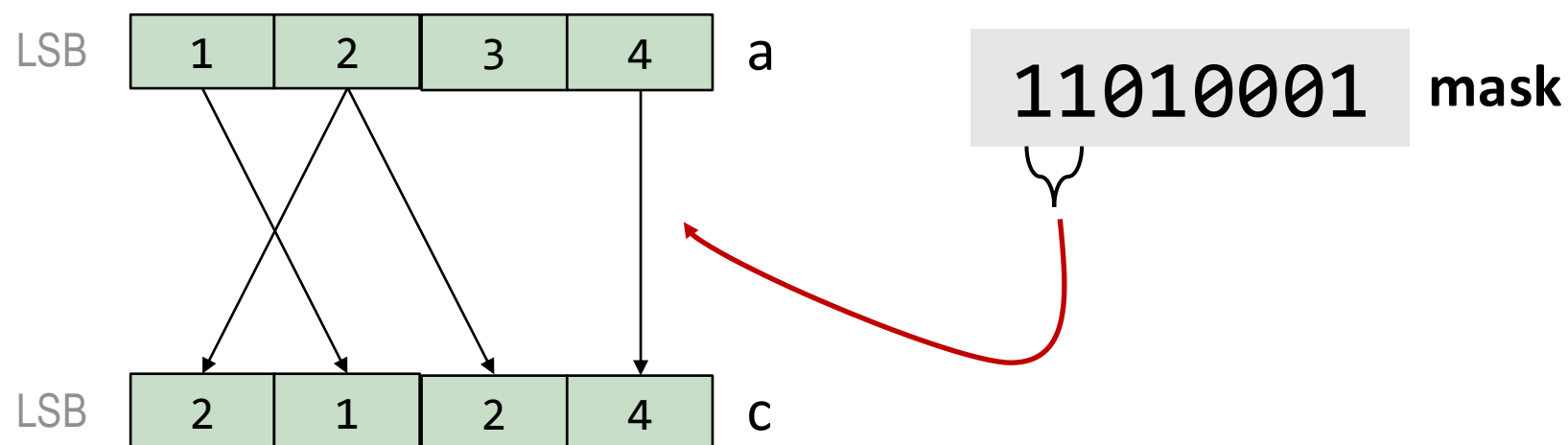
Does not cross between 128-bit lanes

Shuffles

```
__m256d _mm256_permute4x64_pd(__m256d a, int mask)
```

Result is filled in each position by any element of a, as specified by mask

Example:



Somewhat more expensive due to shuffle between 128-bits lanes